Juliano Machado da Silva
João Pedro Madruga

Integração entre i-Educar e Moodle por meio de um serviço de integração

Porto Alegre

2026

Juliano Machado da Silva
João Pedro Madruga

Integração entre i-Educar e Moodle por meio de um serviço de integração

Relatório apresentado à disciplina de Engenharia de Software da Pontifícia Universidade Católica do Rio Grande do Sul como parte das atividades de desenvolvimento do trabalho de integração entre os sistemas i-Educar e Moodle.

Pontifícia Universidade Católica do Rio Grande do Sul – PUCRS

Curso de Engenharia de Software

Orientador: Jose Pedro Schardosim Simão

Porto Alegre
2026

Juliano Machado da Silva
João Pedro Madruga
Integração entre i-Educar e Moodle por meio de um serviço de integração / Juliano Machado da Silva, João Pedro Madruga. – Porto Alegre, 2026.
18 p.

Orientador: Jose Pedro Schardosim Simão

Relatório de trabalho acadêmico – Pontifícia Universidade Católica do Rio Grande do Sul – PUCRS
Curso de Engenharia de Software, 2026.

1. Integração de sistemas. 2. i-Educar. 3. Moodle.

Relatório dedicado às pessoas que contribuíram para a realização deste trabalho.

Agradecimentos

Agradecemos aos professores e colegas que contribuíram para a definição da arquitetura, para a implementação e para a validação da integração entre os sistemas.

Resumo

Este relatório apresenta o desenvolvimento de uma integração entre o i-Educar, sistema de gestão educacional, e o Moodle, ambiente virtual de aprendizagem. A solução utiliza um serviço de integração desenvolvido com NestJS e TypeScript, organizado como um monólito modular com princípios de Arquitetura Hexagonal. O primeiro caso de uso implementado foi a sincronização de turmas: uma turma ativa do i-Educar é identificada, transformada pelo adapter acadêmico e correspondida a um curso no Moodle por meio de Web Services. O serviço mantém um banco PostgreSQL próprio para registrar os vínculos entre identificadores e o histórico das sincronizações. A execução pode ocorrer automaticamente em intervalos regulares ou ser disparada por uma requisição HTTP. O teste ponta a ponta confirmou que uma nova turma criada no i-Educar foi convertida em um curso no Moodle, sem falhas e com o vínculo persistido.

Palavras-chave: integração de sistemas; i-Educar; Moodle; NestJS; Arquitetura Hexagonal.

Abstract

This report presents an integration between i-Educar, an educational management system, and Moodle, a virtual learning environment. The solution uses a NestJS and TypeScript integration service organized as a modular monolith with Hexagonal Architecture principles. The first implemented use case was class synchronization: an active class from i-Educar is identified, transformed by an academic adapter and mapped to a Moodle course through Web Services. The service uses its own PostgreSQL database to store identifier mappings and synchronization history. Execution can occur periodically or be triggered by an HTTP request. The end-to-end test confirmed that a new class created in i-Educar was converted into a Moodle course with no failures and with the mapping persisted.

Keywords: systems integration; i-Educar; Moodle; NestJS; Hexagonal Architecture.

Sumário

Introdução 7

1 POSSIBILIDADES DE INTEGRAÇÃO 8
1.1 Turmas e cursos 8
1.2 Alunos e matrículas 8
1.3 Notas e participação 8

2 FONTES E MECANISMOS DE INTEGRAÇÃO 9
2.1 Informações acadêmicas no i-Educar 9
2.2 Cursos no Moodle 9
2.3 Banco próprio da integração 10

3 ARQUITETURA DA SOLUÇÃO 11
3.1 Monólito modular 11
3.2 Arquitetura Hexagonal e adapters 11
3.3 Camada HTTP e agendamento 11

4 PRIMEIRO CASO DE INTEGRAÇÃO 13
4.1 Descrição do caso de uso 13
4.2 Fluxo da integração 13
4.3 Resultado do teste ponta a ponta 14

5 DESAFIOS E DECISÕES 15
5.1 Inicialização do ambiente 15
5.2 Dependências e dados persistentes 15
5.3 Autenticação entre sistemas 15
5.4 Consistência e falhas parciais 15
5.5 Particularidades das plataformas 16
5.6 Compatibilidade com Windows 16

6 CONCLUSÃO 17

REFERÊNCIAS 18

Introdução

Instituições de ensino utilizam diferentes sistemas para administrar informações acadêmicas e oferecer atividades de aprendizagem. O i-Educar (PORTABILIS, [202-]a) concentra dados da gestão escolar, enquanto o Moodle concentra cursos, conteúdos e atividades educacionais. Quando esses sistemas não se comunicam, informações precisam ser repetidas manualmente, o que aumenta o custo operacional e a possibilidade de inconsistências.

O objetivo deste trabalho é projetar e implementar uma integração entre os dois monólitos por meio de uma aplicação intermediária. A primeira funcionalidade escolhida foi a criação automática de cursos no Moodle a partir de turmas ativas cadastradas no i-Educar. A implementação foi executada em containers Docker, permitindo reproduzir os três sistemas e seus bancos de dados em um ambiente local.

O restante deste relatório descreve as possibilidades de integração levantadas, as fontes das informações, a arquitetura adotada, o fluxo implementado, os desafios encontrados e os resultados da validação.

1 Possibilidades de integração

1.1 Turmas e cursos

A primeira possibilidade é sincronizar turmas do i-Educar com cursos do Moodle. O i-Educar é a fonte de verdade para o código, o nome, o ano e a situação da turma. No Moodle, essas informações podem ser representadas por um curso, usando o nome da turma como nome do curso, um identificador curto baseado no código da turma e um *idnumber* estável, no formato `ieducar-turma-`*`codigo`*.

Essa integração reduz o trabalho de criação dos cursos e evita a criação duplicada de cursos quando a sincronização é executada novamente.

1.2 Alunos e matrículas

Uma segunda possibilidade é sincronizar alunos e matrículas. O i-Educar mantém os dados cadastrais dos alunos, suas turmas e suas matrículas. O Moodle poderia receber os usuários correspondentes e matriculá-los nos cursos relacionados às turmas.

Essa segunda possibilidade não foi implementada nesta etapa. Ela exige decisões adicionais sobre identidade dos usuários, criação ou vinculação de contas, tratamento de dados pessoais, atualização de matrículas e remoção de vínculos. Por isso, foi mantida como evolução futura.

1.3 Notas e participação

Também seria possível sincronizar notas ou informações de participação entre os sistemas. Entretanto, esse fluxo possui maior complexidade de negócio, pois seria necessário definir qual sistema é a fonte oficial das notas, como mapear atividades e períodos letivos e como resolver conflitos. Essa possibilidade foi considerada, mas não faz parte do primeiro caso de uso.

2 Fontes e mecanismos de integração

2.1 Informações acadêmicas no i-Educar

As turmas são obtidas por meio da API do i-Educar. O adapter acadêmico realiza uma requisição para buscar turmas ativas do ano corrente e converte a resposta específica da API para o modelo interno de turma. Dessa forma, o restante da aplicação não precisa conhecer os nomes dos campos ou detalhes do protocolo externo.

Na implementação, o serviço consulta os campos necessários para a sincronização: identificador da turma, nome, ano e escola relacionada. O i-Educar continua sendo responsável pela informação acadêmica original; a aplicação intermediária apenas mantém os dados necessários para controlar a sincronização.

No i-Educar, as turmas ficam armazenadas na tabela `pmieducar.turma` do PostgreSQL e são representadas pelo modelo `LegacySchoolClass`, definido no arquivo `app/Models/LegacySchoolClass.php` do repositório oficial (PORTABILIS, [202-]b). O sistema as disponibiliza pela rota `GET /api/school-class`, protegida por token do Laravel Sanctum (LARAVEL, [202-]), gerado pelo comando `php artisan admin:token`. A integração utiliza os filtros que a própria API já oferece: `active=1` (somente turmas ativas), `year_eq` (somente o ano atual), `only` (somente os campos necessários) e `page`. Como a API devolve no máximo 15 turmas por página, o adapter percorre todas as páginas pelo número da página. Nenhuma alteração no código do i-Educar foi necessária para obter esses dados.

2.2 Cursos no Moodle

O Moodle disponibiliza Web Services REST (MOODLE, [202-]). O adapter do Moodle utiliza o token configurado no ambiente para criar os cursos. Para isso, é usada a função `core_course_create_courses`. O curso criado recebe um nome (o nome da turma), um nome curto único (`turma-`*`codigo`*) e um identificador externo (`ieducar-turma-`*`codigo`*), gravado no campo `idnumber`, que permite localizar no Moodle o curso correspondente a cada turma do i-Educar.

Para isso, foi necessário configurar o Moodle: ativar os Web Services e o protocolo REST, criar um serviço externo com as funções liberadas para a integração e gerar um token de acesso. Essa configuração é feita automaticamente pelo script de instalação. O Moodle pode responder a erros com status HTTP 200; por isso, o adapter verifica o conteúdo da resposta para distinguir um curso criado de uma mensagem de erro.

Essa abordagem evita acesso direto às tabelas internas do Moodle. A comunicação

ocorre por meio da API disponibilizada pelo próprio sistema, o que reduz o acoplamento com sua implementação interna.

2.3 Banco próprio da integração

O serviço possui um banco PostgreSQL (POSTGRESQL, [202-]) separado dos bancos do i-Educar e do Moodle, acessado por meio do TypeORM (TYPEORM, [202-]). Esse banco armazena principalmente:

- o vínculo entre o identificador da turma e o identificador do curso;
- o histórico das execuções de sincronização;
- informações necessárias para identificar falhas e reprocessamentos.

O banco intermediário não substitui os sistemas de origem. Ele funciona como registro de integração e controle operacional.

3 Arquitetura da solução

3.1 Monólito modular

O serviço foi implementado como um monólito modular com NestJS (NESTJS, [202-]a) e TypeScript (MICROSOFT, [202-]b). Os módulos separam responsabilidades relacionadas ao domínio acadêmico, à plataforma de aprendizagem e à sincronização. Essa escolha mantém a implantação simples nesta etapa, sem introduzir a complexidade de comunicação entre vários microsserviços.

Apesar de ser executada como uma única aplicação, a solução possui limites explícitos entre os módulos. Essa organização também facilita uma futura extração do módulo de sincronização para um microsserviço independente.

3.2 Arquitetura Hexagonal e adapters

A integração foi organizada segundo o princípio de Ports and Adapters. As portas representam contratos internos para consultar dados acadêmicos e manipular cursos na plataforma de aprendizagem. Os adapters implementam esses contratos usando as APIs concretas do i-Educar e do Moodle.

Essa separação também funciona como uma *Anti-Corruption Layer* (EVANS, 2003; MICROSOFT, [202-]a): os modelos externos são convertidos para objetos do domínio interno. Assim, uma mudança na API de um dos sistemas tende a ficar concentrada no adapter correspondente.

3.3 Camada HTTP e agendamento

O controller HTTP expõe o endpoint:

```
POST /sincronizacao/turmas
```

Nessa camada é aplicado o padrão MVC: o controller recebe a requisição e chama o caso de uso, sem conter regras de negócio; o modelo corresponde ao caso de uso e aos objetos do domínio; e a visão é a resposta em JSON com o resumo da sincronização. Na Arquitetura Hexagonal, esse controller atua como um adapter de entrada. Assim, o MVC organiza a interação HTTP, enquanto a Arquitetura Hexagonal organiza a aplicação e suas dependências.

Além do disparo manual, um agendamento (NESTJS, [202-]b) executa a sincronização a cada minuto. A sincronização imediata, no momento em que a turma é criada, exigiria que o i-Educar notificasse a integração, o que dependeria de alterar o código do i-Educar. Por isso, optou-se por consultar periodicamente a API já existente; com o intervalo de um minuto, o curso aparece no Moodle praticamente logo após a criação da turma. O caso de uso consulta as turmas, verifica quais já possuem vínculo, cria os cursos faltantes no Moodle e registra o resultado da operação.

4 Primeiro caso de integração

4.1 Descrição do caso de uso

O caso de uso implementado é:

> Quando uma turma ativa do ano atual for identificada no i-Educar, criar o curso correspondente no Moodle, caso ele ainda não exista, e registrar o vínculo entre o identificador da turma e o identificador do curso.

Antes de criar um curso, o serviço consulta a tabela de vínculos do seu próprio banco (`vinculo_turma_curso`). Turmas que já possuem vínculo são ignoradas, o que torna a operação idempotente: executar a sincronização várias vezes não cria cursos duplicados. O identificador externo `ieducar-turma-`*`codigo`* é gravado em cada curso para que ele possa ser localizado no Moodle a partir da turma de origem.

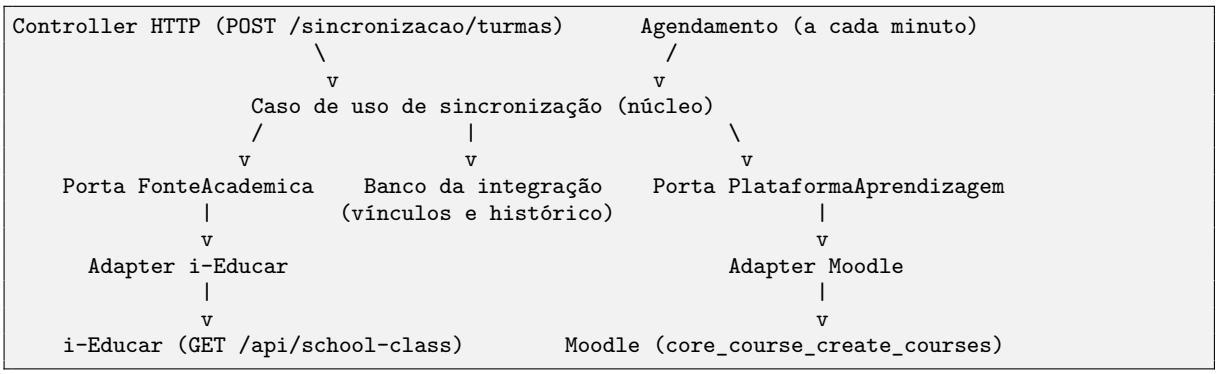
4.2 Fluxo da integração

O fluxo executado pelo serviço é:

1. o agendador ou o endpoint HTTP inicia uma sincronização;
2. o adapter do i-Educar consulta as turmas ativas do ano corrente;
3. os dados externos são convertidos para o modelo interno de turma;
4. o caso de uso consulta os vínculos já registrados;
5. turmas sem vínculo são enviadas ao adapter do Moodle;
6. o Moodle cria o curso correspondente;
7. o serviço persiste o vínculo entre turma e curso;
8. a execução é registrada com totais, sucessos e falhas.

A Figura 1 apresenta esse fluxo. O caso de uso, no núcleo da aplicação, chama cada adapter separadamente por meio de sua porta; os adapters do i-Educar e do Moodle não se comunicam entre si.

Figura 1 – Fluxo da integração entre i-Educar, serviço de integração e Moodle.



Fonte: Elaborado pelos autores.

4.3 Resultado do teste ponta a ponta

O teste automatizado criou uma cópia da turma de identificador 3 no i-Educar. A nova turma recebeu o identificador 24. Em seguida, a sincronização foi disparada manualmente. O resultado foi:

```
totalTurmas: 22
cursosCriados: 1
turmasIgnoradas: 21
falhas: []
```

O curso correspondente recebeu o identificador 23 no Moodle e o banco da integração registrou:

```
turma 24 -> curso 23
```

Ao final, o teste removeu a turma, o curso e o vínculo criados. Portanto, a validação confirmou o fluxo completo entre os dois sistemas e também verificou que as informações persistidas são consistentes.

5 Desafios e decisões

5.1 Inicialização do ambiente

Os três sistemas possuem dependências, bancos e configurações próprias. Para reduzir a diferença entre ambientes, foi utilizado Docker Compose (DOCKER, [202-]). O Moodle e o seu banco MariaDB são executados a partir das imagens da Bitnami (BITNAMI, [202-]b; BITNAMI, [202-]a). O script de instalação cria os arquivos de ambiente, sobe os containers, instala as dependências, executa as migrations e configura os tokens das APIs.

5.2 Dependências e dados persistentes

O i-Educar depende das bibliotecas PHP instaladas pelo Composer no diretório `vendor`; por isso, a instalação dessas dependências faz parte do script de instalação e ocorre antes da execução das migrations. Além disso, os tokens armazenados no arquivo `.env` precisam corresponder à instalação atual dos bancos. A recriação de volumes pode invalidar tokens antigos, exigindo sua geração novamente.

5.3 Autenticação entre sistemas

A comunicação com as duas APIs é autenticada por tokens. No i-Educar, a rota `GET /api/school-class` é protegida pelo Laravel Sanctum, e o token é gerado pelo comando `php artisan admin:token`. No Moodle, o token é vinculado ao serviço externo criado para a integração, que libera apenas as funções utilizadas por ela. O script de instalação gera os dois tokens e os grava no arquivo `.env`, de onde o serviço de integração os lê; assim, nenhum token fica fixo no código. Como cada token pertence ao banco do sistema que o emitiu, ele precisa ser gerado novamente sempre que os volumes forem recriados, conforme descrito na Seção 5.2.

5.4 Consistência e falhas parciais

Uma sincronização pode concluir algumas operações e falhar em outras. Por isso, a resposta do endpoint apresenta totais separados para cursos criados, turmas ignoradas e falhas. O histórico permite acompanhar o que ocorreu em cada execução. Em uma evolução futura, podem ser adicionadas tentativas automáticas, backoff, alertas e uma fila de mensagens.

5.5 Particularidades das plataformas

Durante a implementação, algumas particularidades das plataformas exigiram ajustes. A versão mais recente da imagem do MariaDB (13.x) mostrou-se incompatível com o Moodle 5.0 (MOODLE, 2025), e foi necessário fixar a versão 11.4. A API do i-Educar devolve no máximo 15 turmas por página, e o link de próxima página que ela fornece perde os filtros aplicados; por isso, o adapter percorre as páginas pelo número. O Moodle, por sua vez, responde aos erros com o status HTTP 200, o que obrigou o adapter a verificar o conteúdo da resposta. Além disso, as variáveis de ambiente do banco da integração utilizam um prefixo próprio (`INTEGRATION_DB_`) para evitar colisões entre os três sistemas no Docker Compose.

5.6 Compatibilidade com Windows

Para permitir a instalação no Windows sem instalar outros programas, foi criado o arquivo `instalar.bat`, que executa o script de instalação por meio do terminal que acompanha o Git para Windows (GIT FOR WINDOWS, [202-]). Também foi adicionado um arquivo `.gitattributes` para impedir que o Git converta as quebras de linha dos scripts para o formato do Windows, o que impediria sua execução dentro dos containers Linux. Por fim, os passos de ajuste de permissões do i-Educar, que não são aceitos nas pastas compartilhadas do Windows, passaram a apenas emitir um aviso, sem interromper a instalação.

6 Conclusão

Este trabalho implementou uma integração funcional entre o i-Educar e o Moodle por meio de um serviço em NestJS e TypeScript. O primeiro caso de uso, criação de cursos a partir de turmas acadêmicas, foi escolhido por possuir valor operacional claro e por permitir validar os principais elementos da arquitetura: consulta a uma API externa, transformação de dados, chamada a uma segunda API, persistência de vínculos e execução periódica.

O teste ponta a ponta demonstrou que a solução atende ao fluxo planejado. Uma turma nova foi criada no i-Educar, transformada em curso no Moodle e relacionada no banco da integração, sem falhas. A separação em módulos, portas e adapters mantém o núcleo da aplicação menos dependente das plataformas externas e cria uma base adequada para futuras funcionalidades.

Como próximos passos, destacam-se a sincronização de alunos e matrículas, o tratamento de atualizações e exclusões de turmas, mecanismos de reprocessamento mais completos (hoje, uma turma que falha é apenas tentada novamente na execução seguinte) e observabilidade mais detalhada. Essas evoluções devem preservar a definição de responsabilidades: o i-Educar como fonte dos dados acadêmicos, o Moodle como ambiente de aprendizagem e o serviço de integração como responsável pelo processamento e pelo controle dos vínculos.

Referências

BITNAMI. *Bitnami package for MariaDB*. [S. l.], [202-]. Docker Hub. Disponível em: <https://hub.docker.com/r/bitnamilegacy/mariadb>. Acesso em: 27 out. 2026. Citado na página 15.

BITNAMI. *Bitnami package for Moodle*. [S. l.], [202-]. Docker Hub. Disponível em: <https://hub.docker.com/r/bitnamilegacy/moodle>. Acesso em: 27 out. 2026. Citado na página 15.

DOCKER. *Docker Compose documentation*. [S. l.], [202-]. Disponível em: <https://docs.docker.com/compose/>. Acesso em: 27 out. 2026. Citado na página 15.

EVANS, E. *Domain-Driven Design*: tackling complexity in the heart of software. Boston: Addison-Wesley, 2003. Citado na página 11.

GIT FOR WINDOWS. *Git for Windows*. [S. l.], [202-]. Disponível em: <https://gitforwindows.org/>. Acesso em: 27 out. 2026. Citado na página 16.

LARAVEL. *Laravel Sanctum*. [S. l.], [202-]. Disponível em: <https://laravel.com/docs/sanctum>. Acesso em: 27 out. 2026. Citado na página 9.

MICROSOFT. *Anti-corruption Layer pattern*. [S. l.], [202-]. Azure Architecture Center. Disponível em: <https://learn.microsoft.com/en-us/azure/architecture/patterns/anti-corruption-layer>. Acesso em: 27 out. 2026. Citado na página 11.

MICROSOFT. *TypeScript Documentation*. [S. l.], [202-]. Disponível em: <https://www.typescriptlang.org/docs/>. Acesso em: 27 out. 2026. Citado na página 11.

MOODLE. *Moodle 5.0 release notes*. [S. l.], 2025. Disponível em: <https://moodledev.io/general/releases/5.0>. Acesso em: 27 out. 2026. Citado na página 16.

MOODLE. *Web services*. [S. l.], [202-]. Moodle Docs. Disponível em: <https://docs.moodle.org/dev/Web_services>. Acesso em: 27 out. 2026. Citado na página 9.

NESTJS. *Documentation*. [S. l.], [202-]. Disponível em: <https://docs.nestjs.com/>. Acesso em: 27 out. 2026. Citado na página 11.

NESTJS. *Task Scheduling*. [S. l.], [202-]. Disponível em: <https://docs.nestjs.com/techniques/task-scheduling>. Acesso em: 27 out. 2026. Citado na página 12.

PORTABILIS. *i-Educar*: sistema de gestão educacional. [S. l.], [202-]. Disponível em: <https://ieducar.org/>. Acesso em: 27 out. 2026. Citado na página 7.

PORTABILIS. *i-Educar*. [S. l.], [202-]. Repositório oficial. Disponível em: <https://github.com/portabilis/i-educar>. Acesso em: 27 out. 2026. Citado na página 9.

POSTGRESQL. *PostgreSQL Documentation*. [S. l.], [202-]. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 27 out. 2026. Citado na página 10.

TYPEORM. *Documentation*. [S. l.], [202-]. Disponível em: <https://typeorm.io>. Acesso em: 27 out. 2026. Citado na página 10.